

Batched tensor recursion for the exact discrete Fourier transform: a power saving of 607/1250000 in the logarithm

eumemic

with substantial assistance from Claude (Anthropic)

9 October 2026

Abstract

We show that the discrete Fourier transform of every length n can be computed exactly in

$$O(n(\log n)^\theta (\log \log n)^{4-\theta}), \quad \theta = 1 - \frac{607}{1250000} = 1 - 4.856 \times 10^{-4},$$

operations, in the exact complex-arithmetic model of OpenAI's *An explicit power saving for the exact discrete Fourier transform* (25 September 2026). There the saving $1 - \theta$ is about 2.1×10^{-13} . Two changes give the improvement. First, a frame transition of residual dimension ρ is executed as one recursive call to $C^{\otimes \rho f}$ instead of ρ separate directional steps. Second, the fixed finite network is replaced by the complex supplier of the community repository **CrocSwap/integer-mult-bounds**. That network was built by many contributors, credited in Section 9. The rest of OpenAI's argument is used unchanged. Section 5 (added by Chafik Boukhalfa, with Anthropic Claude assistance) replaces the network by the complex supplier of that repository's PR #200 and obtains $\theta = 1 - 3327/5000000 = 1 - 6.654 \times 10^{-4}$; Section 6 uses its source-assisted supplier (PR #184/#194) and obtains $\theta = 1 - 7009/10^7 = 1 - 7.009 \times 10^{-4}$, and with that word's physical layer replaced by PR #200's (PR #233), $\theta = 1 - 7086/10^7$. The same bound holds for exact complex convolution as a bilinear algorithm, with general multiplication allowed for the pointwise products as in OpenAI's Corollary 1.3. The theorem depends on the correctness of that network. Its proofs are written arguments, checked in part by scripts, and have had no independent human review (Section 8).

1 Result

We use the model of [1, §1.1] without change: exact complex arithmetic at unit cost, input-independent scalars, a specified Fourier root, and unit-cost integer operations on $O(\log(n+2))$ -bit words. Scalar preparation, schedule construction, index work and data movement are charged. Throughout,

$$C = \frac{1}{2} \begin{pmatrix} 1+i & 1-i \\ 1-i & 1+i \end{pmatrix}$$

is the matrix of [1, (2.1)].

Theorem 1.1. *There is a deterministic algorithm that, for every $n \geq 1$ and every $x \in \mathbb{C}^n$, computes $F_n x$ exactly in $O(n(\log n)^\theta (\log \log n)^{4-\theta})$ operations, where $\theta = 1 - 607/1250000$. The same bound holds for the linear convolution of two vectors in $\mathbb{C}^n$ by a bilinear algorithm (not an input-linear circuit), with general field multiplication allowed for the pointwise products, as in [1, Corollary 1.3].*

Corollary 1.2. *For every fixed $\kappa < 607/1250000$, both problems take $O(n(\log n)^{1-\kappa})$ operations; for example, $\kappa = 4.85 \times 10^{-4} > 2^{-11.01}$.*

In [1] the exponent comes from one fixed network through [1, Theorem 2.6]: $C^{\otimes k}$ costs $O(2^k(k+1)^{\theta_0})$ with $\theta_0 = \log_m \lambda$ and $\lambda = m - \Delta/W_*$. In that recurrence every unit of residual dimension is a separate directional step on a fiber of f bits, so it pays one full level of recursion. Here a residual of dimension ρ is a single call on ρf bits (Theorem 3.1). The critical exponent then solves the moment equation $\sum_\rho N_\rho (\rho/m)^\theta = W$. The same batched accounting is used for integer multiplication in [5], where it was introduced by icekylinx (“projector batching” and “whole-residual complex recursion”). Section 7 explains why the remaining sections of [1] apply unchanged.

2 Batched networks

Write an array address as f columns of m bits, followed by spectator bits. A *columnwise* map applies the same map to every column.

Definition 2.1. A *batched network* of width m on W roles is a finite list of operations on W role arrays and finitely many temporary arrays. It must have the following property for every $f \geq 1$ and every spectator set: for arbitrary initial role arrays $A_1, \dots, A_W$, with temporaries starting at zero, the list leaves every role array equal to $C^{\otimes mf} A_w$, tensored with the identity on spectators. Each operation is one of the following.

1. A *gate*: a fixed linear map with coefficients in $\mathbb{Q}(i)$, applied pointwise across a fixed set of arrays (the same address in each array). Copying an array, erasing a temporary and renaming arrays are gates.
2. An *adapter* on one array: a columnwise invertible affine binary address map, followed by multiplication by $\omega^f i^{q(x)}$, where q is a sum over columns of one fixed function $\mathbb{F}_2^m \rightarrow \mathbb{Z}/4$ and $\omega \in \mathbb{Q}(i)$ is a fixed constant (a per-column scalar).
3. A *child* of width $\rho \in \{1, \dots, m\}$ on one array: $C^{\otimes \rho}$ or $C^{-\otimes \rho}$ on ρ fixed bit positions of every column, that is, $C^{\pm \otimes \rho f}$ along those ρf bits.

Let N_ρ be the number of children of width ρ .

Children on temporaries are counted in N_ρ , but temporaries are not roles. This is the usual rank deficit: $\sum_\rho \rho N_\rho < Wm$ is necessary for a saving, and copies of shared values spend part of it.

The property for every f is not automatic. A list that is correct for $f = 1$ can fail for larger f . For example, copy A to T , apply a child P to T , and subtract T from A . This gives $I - P^{\otimes f}$, not $(I - P)^{\otimes f}$. The following condition rules this out.

Lemma 2.2. Give every array a frame, an invertible operator on $\mathbb{C}^{\mathbb{F}_2^m}$ with entries in $\mathbb{Q}(i)$.

- Roles start at the identity, and a temporary created by a copy takes the frame of its source.
- An adapter or child replaces the frame R of its array by XR , where X is its one-column operator. An adapter’s operator includes its per-column scalar ω .

Suppose every gate touches only arrays whose current frames are equal as operators, and every temporary ends at zero. Let S be the matrix of the scalar network obtained by deleting all adapters and children, and let R_w be the final frame of role w . Then for every f the list maps $A_w \mapsto \sum_{w'} S_{ww'} R_{w'}^{\otimes f} A_{w'}$. If for $f = 1$ it maps $A_w \mapsto C^{\otimes m} A_w$ for every role, then for every f it maps $A_w \mapsto S_{ww}^{1-f} C^{\otimes mf} A_w$. A final adapter with $\omega = S_{ww}$ (identity address map, $q = 0$) followed by the fixed gate S_{ww}^{-1} multiplies role w by S_{ww}^{f-1} , which gives the property of Definition 2.1 for every f .

Proof. By induction over the list, each array a holds $R_a^{\otimes f} \sum_{w'} S_{aw'}^{(t)} A_{w'}$, where $S^{(t)}$ is the scalar network so far. An adapter or child applies $X^{\otimes f}$ and so multiplies the frame. A gate acts on arrays that hold $R^{\otimes f} v_j$ with the same R . It maps them to $R^{\otimes f} \sum_j g_{ij} v_j$, because a pointwise scalar map commutes with an operator applied identically to every touched array. A copy duplicates $R^{\otimes f} v$. For $f = 1$ the hypothesis gives $S_{ww'} R_w = \delta_{ww'} C^{\otimes m}$. Since R_w is invertible, $S_{ww'} = 0$ for $w' \neq w$ and $R_w = S_{ww}^{-1} C^{\otimes m}$. Hence $S_{ww} R_w^{\otimes f} = S_{ww}^{1-f} C^{\otimes mf}$. Temporaries are handled in the same way. $\square$

Lemma 2.3. *Let $k = mf + r$ with $0 \leq r < m$. Fix an adapter as in Definition 2.1 and a set of ρ bit positions in a column. Applying the adapter to an array of length 2^k , and then moving the ρ selected bits of every column into one contiguous least-significant field of ρf bits, takes $O(2^k)$ time. This includes index preparation and data movement. The constant depends on m and the fixed maps, but not on f or k . The inverse movement has the same bound.*

Proof. This is the proof of [1, Lemma 2.5] with two additions. For every column digit, the finite table also records the selected ρ bits and the value of the phase function on that column. The prefix traversal carries a third partial value, the accumulated phase in $\{\pm 1, \pm i\}$. Extending a prefix by a digit multiplies it by the tabulated phase in constant time. At a leaf the value is copied between the two computed addresses and multiplied by the accumulated phase and by ω^f , which is prepared once in $O(f)$ operations. The selected field has ρf bits, so each fiber of the following child is contiguous, and W consecutive fibers form a contiguous block of $W2^{\rho f}$ entries. $\square$

3 The batched recursion

Theorem 3.1. *Let $\mathcal{N}$ be a batched network of width m on W roles whose children all have width at most $m - 1$. If $0 < \theta < 1$ and*

$$\mu := \frac{1}{W} \sum_{\rho} N_{\rho} \left(\frac{\rho}{m} \right)^{\theta} < 1,$$

then $C^{\otimes k}$ can be applied exactly to an arbitrary complex array of length 2^k in $O(2^k(k+1)^{\theta})$ time, for every $k \geq 0$. The bound includes fixed-table construction, all scalar arithmetic, index preparation and data movement. Only constants in $\mathbb{Q}(i)$ are used.

Proof. We describe a procedure $\mathcal{A}(k)$ that applies $C^{\otimes k}$ to each of W arbitrary arrays of length 2^k , and bound its cost $T(k)$. Put $t(k) = T(k)/(W2^k)$. Let $\rho_{\max} \leq m - 1$ be the largest child width of $\mathcal{N}$.

For $k < K$, with K fixed below, apply the k factors of C one at a time along their fibers. This costs $O(kW2^k)$, so $t(k) \leq c_1 K$.

For $k \geq K$ write $k = mf + r$ with $0 \leq r < m$. Apply the r spectator factors directly, at cost $O(rW2^k)$. Then execute $\mathcal{N}$ on the W arrays with f columns. Gates cost $O(2^k)$ per array touched. Adapters cost $O(2^k)$ by Lemma 2.3. A child of width ρ on one array first moves its selected bits into the low field (Lemma 2.3), so that the array consists of $2^{k-\rho f}$ contiguous fibers of length $2^{\rho f}$. Group the fibers into $\lceil 2^{k-\rho f}/W \rceil$ batches of W fibers, padding the last batch with zero arrays, and run $\mathcal{A}(\rho f)$ on each batch. Then undo the movement. An inverse child is a forward child followed by a translation [1, Lemma 2.2]. Correctness follows by induction on k , because $\rho f < k$ and Definition 2.1 holds for arbitrary role values. Zero padding is mapped to zero. Each call allocates and zeroes its own temporaries, at cost $O(2^k)$ per temporary.

There is a constant A_0 , depending only on $\mathcal{N}$ and m , with

$$T(k) \leq A_0 W 2^k + \sum_{\text{children } c} \left(\frac{2^{k-\rho_c f}}{W} + 1 \right) T(\rho_c f).$$

Since $k - \rho_c f \geq (m - \rho_{\max})f \geq f$, dividing by $W 2^k$ gives

$$t(k) \leq A_0 + (1 + W 2^{-f}) \sum_{\rho} \frac{N_{\rho}}{W} t(\rho f).$$

Suppose $t(j) \leq B(j+1)^{\theta}$ for all $j < k$. Since $\rho f + 1 \leq (\rho/m)(k+1) + 1 \leq (\rho/m)(k+1)(1 + m/(k+1))$,

$$t(k) \leq A_0 + B(k+1)^{\theta} \mu (1 + W 2^{-f}) \left(1 + \frac{m}{k+1} \right).$$

Choose K so large that the product of μ and the last two factors is at most $\mu' = (1 + \mu)/2 < 1$ for all $k \geq K$, and put $B = \max\{c_1 K, A_0/(1 - \mu')\}$. Then $t(k) \leq B(k+1)^{\theta}$ for every k . Finally, place the input in one role, set the others to zero, and run $\mathcal{A}(k)$. Since W is a constant, the cost is $O(2^k(k+1)^{\theta})$. Index widths, the recursion stack and workspace are bounded exactly as in the proof of [1, Theorem 2.6]: the array lengths shrink geometrically, and addresses have $k + O(1)$ bits. $\square$

Remark 3.2. [1, Theorem 2.6] corresponds to the case where every child has width one, up to the strict inequality: then $\mu = S/(Wm^{\theta})$ and the condition reads $\theta > \log_m(S/W)$, whereas [1, Theorem 2.6] reaches $\theta = \log_m(S/W_*)$ exactly with a padded role count W_* . In Theorem 3.1 a child of width ρ costs $(\rho/m)^{\theta}$ of a role, not $\rho(1/m)^{\theta}$. To first order in $1 - \theta$, the critical saving is the rank deficit $Wm - \sum_{\rho} \rho N_{\rho}$ divided by $\sum_{\rho} N_{\rho} \rho \ln(m/\rho)$, rather than by $\ln m \sum_{\rho} \rho N_{\rho}$. Power-of-two padding of the role count is not needed, because partial batches are lower order. Theorem 3.1 also requires the network property for arbitrary role values, for the reason given in [1, Remark 2.7].

4 The network

We first record why the child histogram of a network depends only on binary frame data. Write $\mathcal{P}$ for the Pauli group generated by translations $X_a f(x) = f(x + a)$ and signs $Z_b f(x) = (-1)^{b \cdot x} f(x)$ on arrays indexed by $\mathbb{F}_2^m$. An invertible operator U that normalizes $\mathcal{P}$ induces a binary symplectic map g_U on Pauli labels. Let $\rho(U)$ be the rank of its block that maps sign labels to translation labels. As in `notes/general-clifford-frames.tex` of [5], this is the Fourier rank of a frame transition.

Lemma 4.1. *Let U normalize $\mathcal{P}$ and have entries in $\mathbb{Q}(i)$, and put $\rho = \rho(U)$. Then $U = u A_1 (C^{\otimes \rho} \otimes I) A_2$. Here A_1, A_2 are adapters in the sense of Definition 2.1 with $f = 1$ and $\omega = 1$, $C^{\otimes \rho}$ acts on ρ coordinate bits, and $u \in \mathbb{Q}(i)$ is a scalar. The width ρ is determined by g_U alone.*

Proof. Let $P \subset \text{Sp}(2m, \mathbb{F}_2)$ be the stabilizer of the sign labels. By the Bruhat decomposition for this parabolic, $\text{Sp}(2m, \mathbb{F}_2)$ is the disjoint union of the double cosets $P w_{\rho} P$, $0 \leq \rho \leq m$, where w_{ρ} exchanges translation and sign labels on ρ coordinates. The rank $\rho(\cdot)$ is constant on each double coset. Every element of P is induced by an adapter: invertible address maps $x \mapsto Mx$ induce the Levi part, and diagonal phases $i^{x^T \tilde{S} x}$ with $\tilde{S}$ an integer symmetric lift induce the symmetric shears. The operator $C^{\otimes \rho}$ on ρ coordinates induces an element of $P w_{\rho} P$, since C exchanges the two labels of its coordinate up to a shear. Hence $g_U = p_1 g_{C^{\otimes \rho}} p_2$ with $p_1, p_2 \in P$. Choosing adapters A_1, A_2 that induce p_1, p_2 , the operator $V = A_1 C^{\otimes \rho} A_2$ induces g_U . Then UV^{-1} maps every Pauli to itself up to sign, so it is a scalar multiple of a Pauli. A Pauli is an adapter, so it is absorbed into A_1 . The scalar lies in $\mathbb{Q}(i)$ because U and V have entries in $\mathbb{Q}(i)$. $\square$

Consequently, errors in the exact phases or Pauli corrections of frame representatives can only change adapters and scalars. They cannot change the child histogram. For a columnwise operator $U_0^{\otimes f}$ the lemma applies to U_0 , which gives columnwise adapters and the scalar u^f .

Proposition 4.2. *The paired-cube complex supplier of [5] at commit `d1d6c07` is a batched network of width $m = 72$ on $W = |G| \cdot 29937$ roles, with $G = O(72, 2)$ and $N_\rho = |G| \cdot n_\rho$. Here n_ρ is the per-vertex child histogram in `certificates/network-children.json`. Every child has width at most 60.*

Proof (by reference). The construction and its proofs are in [5], pinned at commit `d1d6c07` (full hash in the bibliography). The entry point is `notes/paired-cube-note.tex` and the files it includes. We use four facts.

1. The completed three-stage word with shared completed cores restores the logical value of every auxiliary role and exchanges the two data banks. Each role therefore physically undergoes the exact $C^{\otimes m}$, up to fixed adapters, signs, a fixed Pauli translation and the bank exchange. Sources: `notes/paired-cube-construction.tex` (the identity $K + H + B = I$, the dirty-word contract, copied centers), `notes/paired-cube-sharing.tex` (sequential completed-core sharing: no role is used by two invocations at once; raw operator contract; final gauge child), and `notes/three-stage-cover-complex.tex` (chronology and phase endpoints). These notes in turn rely on the positive producer of PR #117, the carrier compiler, the copied-center and deferred old-value interface, and frozen gauge-witness files, all recorded in [5].
2. Every frame transition is an exact operator with entries in $\mathbb{Q}(i)$ that normalizes $\mathcal{P}$, so Lemma 4.1 writes it as adapters around one child of width equal to its Fourier rank. The frame representatives and their ranks are given in `notes/general-clifford-frames.tex`. Nondegenerate residuals use the normal form of `notes/endpoint-gauge-complex.tex`.
3. Copied centers are temporary streams: copy, transform, read, erase (`notes/copied-centers-lemma.tex`). Their transforms are children.
4. The complete child multiset per cover vertex is the ledger of `notes/paired-cube-sharing.tex`, §“The complete paid ledger”, instantiated in `certificates/paired-cube-complex-input.json`: $\sum_\rho \rho n_\rho = 2153528 = 72 \cdot 29937 - 1936$, with largest child 60.

All gate coefficients are rational or i , with denominators dividing 6. The star-center scatter, for example, uses the prepared rational scalar $-1/6$, which the model of [1, §1.1] allows.

The network is frame-consistent in the sense of Lemma 2.2: at every common scalar gate it uses one identical exact representative on all incident registers (`notes/general-clifford-frames.tex`, §“Exact representatives and nested children”). Its frames are the representatives T_U of that note, which are defined for every binary subspace U , including the degenerate star frames U_i . We do not use the frames Φ_U of [1, (2.5)], which need U nondegenerate. Fact 1 is the identity for one column, so Lemma 2.2 gives it for every f . Upstream also states the f -column form of each residual directly: one $C^{\otimes rf}$ child, the scalar raised to the power f , and columnwise quadratic phases modulo four (`notes/endpoint-gauge-complex.tex`). Routing between invocations renames arrays. Every multiplicity is $|G|$ times the per-vertex count, so $|G|$ cancels from μ . The data fronts, rank-two complements, width-60 gauge children, copied-center transforms, endpoint Paulis and bank exchange all fit Definition 2.1. $\square$

Proposition 4.3. *For the network of Proposition 4.2 and $\theta = 1 - 607/1250000$,*

$$\sum_{\rho} n_{\rho} \left(\frac{\rho}{72} \right)^{\theta} < 29937 - 0.00315,$$

so $\mu < 1$.

Proof. Write $(\rho/m)^{\theta} = (\rho/m) \exp(a \ln(m/\rho))$ with $a = 607/1250000$. The script `scripts/verify_moment.py` bounds $\ln(m/\rho)$ above by the series $2 \sum y^{2j+1}/(2j+1)$, $y = (x-1)/(x+1)$, with every term rounded up to a 10^{-60} grid and an explicit geometric tail. It also uses $e^x \leq 1 + x + x^2/(2(1-x/3))$ for $0 \leq x < 3$. The sum is evaluated in exact rational arithmetic, and the certified margin is recorded in `certificates/moment.json`. The critical saving of this histogram is $4.85657 \dots \times 10^{-4}$. As a control, the script fails at $a = 4.8566 \times 10^{-4}$. $\square$

This is an independent re-check, and $607/1250000$ is the upstream saving rounded down. The repository [5] certifies the same moment at $a_c = 4856569/10^{10}$ (`notes/paired-cube-assembly.tex`, with normalized gap about 2.0×10^{-10} in `scripts/paired_cube_network.py`). Our script also passes at that value, with margin 6.0×10^{-6} , and either value may be used in Theorem 1.1.

5 A stronger supplier: PR #200

The repository [5] has continued past commit `d1d6c07`. Its later complex suppliers keep the shared-core ledger of Proposition 4.2 and lower the moment. We use the complex supplier of PR #200 ([2]; Chafik Boukhalfa with Anthropic Claude and OpenAI Codex assistance), which is the PR #168 v4 lineage of eumemic (annealed modules, fused outputs, nested schedules, bundle descent, pinned at `4a3c769e`) with that package’s own physical operation frames, 2,310 compensated reuse pairs and 44 terminal sinks (James Chang’s PR #166 technique). Its parameters are $h = 22$, $m = 66$, $v = 1320$, $W = 13,163$ roles per cover vertex, rank deficit $D = 2v - 3\ell = 1320$ with $\ell = 440$, and largest child 20.

Proposition 5.1. *The complex supplier of [2] is a batched network of width $m = 66$ on $W = |G| \cdot 13163$ roles, $G = O(66, 2)$, with $N_{\rho} = |G| \cdot n_{\rho}$, where n_{ρ} is the per-vertex child histogram in `certificates/network-children-pr200.json` (`certificate.json`, field `complex.profile`, of that package). Every child has width at most 20.*

Proof (by reference). The four facts of the proof of Proposition 4.2 hold with the following sources, all pinned in [2].

1. The completed three-stage word with shared completed cores is retained from PR #144 unchanged (`PROOF.md`, “Paid balanced transfer”, and the retained notes `paired-cube-sharing.tex`, `three-stage-cover-complex.tex`). The local scalar word differs: it is the annealed, fused word of PR #168 v4 with the package’s local circuit. Its complete signed scalar identity is replayed by the package verifier on every formal source, target and dirty column of one column, in both shear signs (`complex/physical.py`, `complex/verify\_fused.py`), together with the carrier closure, centre closure and target reads. This is the one-column identity that Lemma 2.2 extends to every f .
2. Every operation carries one rational frame shared by both of its registers (the “operation frame”), and every role’s frames nest along its chronology: its entrance (zero, its source line or its gauge), its operations’ frames, its root frame if it is a root, and the full space. The

frames are the exact representatives of `general-clifford-frames.tex`, vendored in that package, so Lemma 4.1 writes each transition as adapters around one child of width equal to its Fourier rank, which is the rank increment along the chain. The verifier checks every nesting and every rank (`complex/physical.py`).

3. The constructions beyond PR #144 are operations of Definition 2.1. A compensated reuse pair renames a dead role’s array to its recipient after a compensation read of the inherited dirty value with exact adjoint coefficients (gates); the recipient’s gauge is read at the handoff, and the step into it is counted as a child. A terminal sink deletes a destination-only register and routes its writes through a pivot target (gates), removing one child of width r and one of width $h - r$ per sink and stage; the package re-counts the ledger after the deletion and replays the modified word. Fused outputs merge same-singleton signed outputs on disjoint supports (gates). Copied centres are temporary streams as before.
4. The complete per-vertex child multiset is the ledger of `paired-cube-sharing.tex` re-counted by `complex/physical.py` and recorded in `certificate.json`: $\sum_{\rho} \rho n_{\rho} = 867438 = 66 \cdot 13163 - 1320$, largest child 20. The verifier also certifies the moment root $a_c = 665489485337/10^{15}$ with its own outward interval arithmetic and rejects the next 10^{-15} grid point.

All gate coefficients are rational or i : signed shears have coefficients ± 1 , decoder denominators divide six, and dirty-value compensation uses exact rational adjoint coefficients. The network is frame-consistent at every scalar gate by construction of the operation frames. The retained all-size contracts of [2] that concern the network itself (general Clifford lifts, common generic bases, uniform weighted compilation and completed exact complex tensor contracts) are the same kind of written arguments as for Proposition 4.2; its remaining contracts (arbitrary-width routing, restored internal rows, layout, prime supply, precision/recovery, uniform setup and analytic fixed-tape transfer) belong to the multiplication assembly and are not used here, as in Section 8. $\square$

Proposition 5.2. *For the network of Proposition 5.1 and $\theta = 1 - 3327/5000000$,*

$$\sum_{\rho} n_{\rho} \left(\frac{\rho}{66} \right)^{\theta} < 13163 - 0.00269,$$

so $\mu < 1$.

Proof. The script `scripts/verify\_moment.py` with `--childrencertificates/network-children-pr200.json` evaluates the same upper bounds as in Proposition 4.3 in exact rational arithmetic; the certified margin is recorded in `certificates/moment-pr200.json`. As a control, the script fails at $a = 6655/10^7$. The upstream certified value $a_c = 6.65489 \dots \times 10^{-4}$ lies between the two; the bounds used here are slightly weaker than upstream’s, so $3327/5000000 = 6.654 \times 10^{-4}$ is the value we certify. $\square$

Theorem 5.3. *Theorem 1.1 holds with $\theta = 1 - 3327/5000000 = 1 - 6.654 \times 10^{-4}$. For every fixed $\kappa < 3327/5000000$, both problems take $O(n(\log n)^{1-\kappa})$ operations.*

Proof. Replace Propositions 4.2 and 4.3 by Propositions 5.1 and 5.2 in Section 7; nothing else changes. The width $m = 66$ enters only through the moment and the base range. $\square$

6 The source-assisted supplier: PR #194

The frontier complex supplier of [5] at the time of writing is the source-assisted word of icekylinx (PR #184, with OpenAI GPT-6 Astra assistance) rebuilt on the PR #168 v4 modules by ikeboy (PR #193/#194, [3]). It is a different compilation of the same paired-cube program: equal-frame fresh-channel compression (“frame flow”) with exact arbitrary-dirty lifts. Its parameters are $h = 22$, $m = 66$, $v = 1320$, $W = 12,052$ roles per cover vertex, deficit 1320 and largest child 20.

Proposition 6.1. *The complex supplier of [3] is a batched network of width $m = 66$ on $W = |G| \cdot 12052$ roles, with $N_\rho = |G| \cdot n_\rho$ and n_ρ the per-vertex child histogram in `certificates/network-children-pr194.json` (field `complex\_profile` of its `certificate.json`). Every child has width at most 20.*

Proof (by reference). The construction is `notes/source-assisted-note.tex` of [5] (PR #184) with the v4 package `research/source-assisted-v4` of [3]; its checkers are `complex\_frame\_flow.py`, `exact\_complex\_flow\_lift.py` and `contract\_v4.py`.

1. The complete transcript is a linear word $z_{\text{final}} = Mz + Lx$ with M invertible: every operation is an exact local map with dyadic coefficients (the lift certificate records all coefficients in $\{\pm 1, \pm 2, \pm 1/2\}$ and their inverse elementary gate programs), applied pointwise, or an original-source control; these are gates. Reads subtract the new dirty response Az at frame zero and leave $y + Bx$ with $K + B = I$ checked exactly; after all reads every live and retired auxiliary is brought to the full frame and every gate is reversed, restoring z . This is the one-column identity, replayed by the independent complex scalar transcript with arbitrary dirty probes and mutation controls; Lemma 2.2 extends it to every f .
2. The flow is monotone along nested frame chains (“later reads follow their certified nested chains”; the lift checker verifies a monotone flow and an acyclic once-only reuse DAG, with frame containment at every vertex). Each frame transition is an exact operator with entries in $\mathbb{Q}(i)$ that normalizes $\mathcal{P}$, so Lemma 4.1 writes it as adapters around one child of width its Fourier rank; the per-vertex child histogram counts these transitions, the births of fresh coordinates through paid containing-frame transitions, the data children and the copied centres, with the same ledger identity $\sum_\rho \rho n_\rho = 794112 = 66 \cdot 12052 - 1320$.
3. The invertible auxiliary maps of the lemma of that note realize all outgoing forms at a frame from the incoming fresh responses plus source controls and retain the complementary coordinates; they are gates on the arrays at that frame, and the compression removes roles, not children.
4. The contract checker certifies the fresh columns, read chronology, target chains and the normalized profile, and [3] certifies the moment root $7.00918 \dots \times 10^{-4}$ with outward intervals.

As for Proposition 4.2, the exact complex phases and the cover are written arguments; the scalar transcript replay checks the one-column identity but does not materialize the physical Clifford transports. The retained all-size interfaces of [3] that concern the multiplication assembly are not used here. $\square$

Proposition 6.2. *For the network of Proposition 6.1 and $\theta = 1 - 7009/10^7$, $\sum_\rho n_\rho (\rho/66)^\theta < 12052 - 0.00052$, so $\mu < 1$. The script fails at $a = 7010/10^7$.*

Theorem 6.3. *Theorem 1.1 holds with $\theta = 1 - 7009/10^7 = 1 - 7.009 \times 10^{-4}$. For every fixed $\kappa < 7009/10^7$, both problems take $O(n(\log n)^{1-\kappa})$ operations.*

Proof. As for Theorem 5.3, with Propositions 6.1 and 6.2. □

Proposition 6.4. *The complex supplier of [4] (PR #194’s word with only its physical layer, operation frames and reuse pairs, replaced by PR #200’s descent) is a batched network of width 66 on $|G| \cdot 12052$ roles with the per-vertex children of `certificates/network-children-pr233.json`; every child has width at most 20. Proof (by reference). The proof of Proposition 6.1 applies verbatim: the flow, exact lift and contract are PR #184/#194’s unchanged on that layer, and PR #200’s complete complex checker admits the layer on the aligned word; the ledger identity $\sum_{\rho} \rho n_{\rho} = 794112 = 66 \cdot 12052 - 1320$ holds with a different child histogram, and [4] certifies the moment root $3543063/5000000000 = 7.0861 \dots \times 10^{-4}$. For $\theta = 1 - 7086/10^7$ the script gives $\sum_{\rho} n_{\rho}(\rho/66)^{\theta} < 12052 - 0.00035$ and fails at $a = 7087/10^7$.*

Theorem 6.5. *Theorem 1.1 holds with $\theta = 1 - 7086/10^7 = 1 - 7.086 \times 10^{-4}$.*

Theorems 5.3, 6.3 and 6.5 rest on different suppliers: the first on the physical-frame ledger of PR #168/#200, whose construction is the closest to Proposition 4.2; the second on the flow compilation of PR #184/#194. Either may be cited on its own.

7 Proof of Theorem 1.1

By Propositions 4.2 and 4.3, Theorem 3.1 gives $C^{\otimes k}$ in $O(2^k(k+1)^{\theta})$ with $\theta = 1 - 607/1250000$, using only rational constants and i , which [1, §5.3] relies on.

The remaining sections of [1] use their Theorem 2.6 through this bound and through two further properties: only constants in $\mathbb{Q}(i)$ are needed, which fixes the root order D_* in their §5, and the word-size accounting of the recursion. Both hold for Theorem 3.1. The exact-width Fourier words of §3, the sector synchronization of §4 ([1, Proposition 4.2], where each width- 2^k sector carries $C^{\otimes k}$), and the small-prime working lengths and chirp convolution of §5 are used unchanged. Their final estimate also uses $\theta > 0$. With our θ their proof of Theorem 1.1 gives $O(n\ell^{\theta}(\log \ell)^4)$, where $\ell = \Theta(\log n / \log \log n)$ is the axis count. This is $O(n(\log n)^{\theta}(\log \log n)^{4-\theta})$. Their proof of Corollary 1.3 uses three transforms of length $2n - 1$, which gives the convolution bound. Their Corollary 1.2 uses their specific constants m, W_*, Δ ; our Corollary 1.2 replaces it. Their Appendix A is not needed.

The fixed network enters only through finite tables and constants. Its role count $W \approx 2^{2570}$ (2571 bits) and the base range K are absolute constants. Because $1 - \mu \approx 10^{-7}$, K is of order 10^9 . These constants are astronomically large: this is an asymptotic existence and explicitness result, with no practical crossover.

8 Scope

This note is a paper proof plus an exact finite certificate. It is not formally verified. The commit of openai/math pinned for [1] (`adc7f12`) has no Lean formalization of its uniform statements. A formalization of the uniform 10^{-13} DFT and convolution statements was added later, at commit 3014888 (8 October 2026), in `lean/ComparatorChallenges/UniformFourier.lean` and `lean/OAI/Computability/FourierTransform/`, with scope in `lean/docs/130.md`. It does not cover the changes here.

Theorem 1.1 depends on Proposition 4.2, whose proof is the construction of [5]. That construction has had maintainer review assisted by OpenAI Codex, but no independent human peer review. Its records state what is checked by script and what is a written argument:

- checked by script: the local scalar identities, including the local scalar map on all $1760^2 = 3,097,600$ source/target entries, binary frame and rank data, and integer replays of the mixer;
- frame checks cover binary geometry, not exact complex phases;
- the exact complex phases, the Cayley cover and completed-core sharing are written proofs, and the cover is not materialized by the verifier.

By Lemma 4.1, the child histogram depends only on the binary frame data.

In [5] the network is one input to an integer-multiplication bound. That bound is conditional on the interfaces stated in `notes/paired-cube-note.tex` there. Each one is either replaced here or not needed:

- Uniform recursion is replaced by Theorem 3.1.
- Ordered-affine streaming of the address work is replaced by Lemma 2.3.
- Analytic reduction, semantic precision and exact recovery concern finite-precision Gaussian resampling and recovery. In exact arithmetic they are not needed, and the reduction from every length to tensor powers of C is that of [1, §§3–5].
- Fixed-tape setup concerns the multitape Turing machine and is not needed in the model of [1, §1.1].
- The opposite-bank primitive, ordinary leaves and prime packing belong to the bit interchange and the multiplication assembly, which are not used.

9 Credits

The finite network is community work recorded in [5]. Its NOTICE, CONTRIBUTORS file and `docs/research/community-round6-review.md` at commit `d1d6c07` give the authoritative attribution, and all of it applies here. The relevant upstream NOTICE blocks are reproduced in this repository’s NOTICE. The principal contributions to this network are:

- **icekylinx** (with OpenAI GPT-6 Astra and Codex assistance): the paired-cube identity $K + H + B = I$, signed overlap channels, the original-source involution, coordinate-star centers, the frame and carrier compiler, and chronological gauges; the arbitrary-subspace Clifford frame theorem and the three-stage Cayley cover; copied retained centers; endpoint gauges; and the batched (whole-residual) recursion that Theorem 3.1 adapts.
- **an664** (with OpenAI Codex assistance): completed-core workspace sharing (PR #128).
- **eumemic** (with Claude assistance): the positive producer DAG whose restrictions supply the coarse query modules (PR #117), and earlier compressed complex networks and source frames (PRs #3, #13).
- **Aurel Prosz (Paureel)**: the copy/transform/read/discard interface and the two-stage topology. **Zhihao Chen (jacklightChen)**: translated complex endpoints and two-stage composition. **Swapnil Jain**: two-stage development. **dleen**: retained totals and stage sharing (PR #4).
- **Douglas Colkitt**: the repository, its compact-control framework, integration and review.

- **OpenAI**: the original complex phase network of *Integer multiplication below $n \log n$* , and the exact-transform framework of [1] reused here.

For the supplier of Section 5: **eumemic** (with Anthropic Claude and OpenAI Codex assistance), PR #117/#155/#157/#161/#168: module infrastructure, scalar modules, physical frames and reuse, output merging, annealed modules, nested schedules, fused outputs and bundle descent; **James Chang (jamesyc)**, PR #124/#166: compensated birth cuts and terminal-output accumulation; **DaysSky**, PR #162/#192: extended carrier closure and the frame-layout ceiling; **GamingPuzzled**, PR #160; **gupt1156** and **Abhinav Ramachandran**, PR #148/#158; **Rohan Arun**, PR #100/#103; **Chafik Boukhalfa** (with Anthropic Claude and OpenAI Codex assistance), PR #175/#179/#181/#189/#196/#200: the package’s local circuits, operation-frame descent, independent complete admission and the fast checker. Section 5 claims only the observation that this supplier is a batched network and the re-check of its moment. Other contributors credited in [5] for parts of this network’s lineage retain that credit. This note claims only the following: the transfer of batched accounting to the exact-DFT model (Lemma 2.3, Theorem 3.1), the normal-form lemma as used here, an independent re-check of the moment, and the observation that the community network applies.

References

- [1] OpenAI, *An explicit power saving for the exact discrete Fourier transform*, 25 September 2026. Repository <https://github.com/openai/math>, commit `adc7f1241b42e322a6451854ab7e4b4c146bf78a`, directory `preprints/An-explicit-power-saving-for-the-exact-discrete-Fourier-transform-September-25-2026`.
- [2] C. Boukhalfa, *A stronger bit supplier: fixed-coordinate and face-diagonal bit local circuit on PR168 v4*, PR #200 of <https://github.com/CrocSwap/integer-mult-bounds>, commit `a1175449f34d39ff933d9d8ab23ced1f32b290ec`, package `research/paired-cube-diagonal-bit-168` (its complex supplier is the one used here).
- [3] ikeboy, *Source-assisted v4 complex word with PR189’s bit supplier*, PR #194 of <https://github.com/CrocSwap/integer-mult-bounds>, commit `a8c87783785be6fe20c4a8bb2ac4b3eec295a179`, package `research/source-assisted-v4`; construction by icekylinx (PR #184, `notes/source-assisted-note.tex`), with OpenAI GPT-6 Astra assistance.
- [4] C. Boukhalfa, *A stronger complex supplier: PR200’s physical layer on the source-assisted v4 word*, PR #233 of <https://github.com/CrocSwap/integer-mult-bounds>, commit `52c6fba58ff74fdca1131e50379287491e599f74`, package `research/source-assisted-v4-layer`.
- [5] D. Colkitt et al., *A sharper exponent for integer multiplication* (community repository), <https://github.com/CrocSwap/integer-mult-bounds>, commit `d1d6c070f5a8c684727ee7ec35d930f9ebfa9758`.